

UNIVERSITAT DE LES ILLES BALEARS

Laboratori de Tecnologies de la Informació i Multimèdia

Práctica 3

WebRTC con servidor de señalización

mediante WebSockets y extensiones de tiempo real

Asignatura	Tecnologías Multimedia e Interacción
Alumno	Sebastià Serra
Curso	2025 – 2026
Entrega	Junio 2026

1. Descripción general de la aplicación

En esta práctica se ha desarrollado una aplicación web de videollamada en tiempo real que permite la comunicación multimedia entre dos navegadores. La aplicación está integrada con el reproductor de vídeo de la Práctica 1, de forma que desde el mismo reproductor se puede acceder a la videollamada a través de un botón en la interfaz.

La aplicación permite que dos usuarios se conecten a una sala virtual, intercambien audio y vídeo en tiempo real, envíen mensajes de texto a través de un canal de datos, y al finalizar la sesión obtengan un resumen generado por inteligencia artificial que incluye la transcripción del audio.

La arquitectura se divide en dos partes bien diferenciadas: la señalización, que se realiza a través de WebSockets mediante Socket.IO, y la transmisión multimedia en tiempo real, que se realiza directamente entre navegadores mediante WebRTC.

2. Relación con la Práctica 1

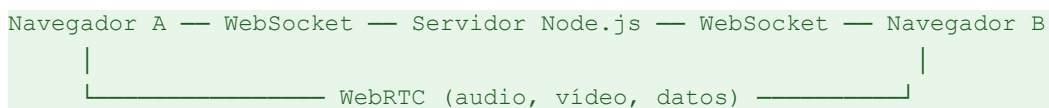
La Práctica 1 consistió en el desarrollo de un reproductor multimedia interactivo con gestión de información semántica mediante pistas VTT. En esta tercera práctica se ha reutilizado y ampliado la misma arquitectura cliente-servidor.

El servidor Node.js con Express que se ha creado para esta práctica sirve también los archivos estáticos del reproductor de la Práctica 1. Ambas aplicaciones comparten el mismo servidor y el mismo dominio: el reproductor está accesible en la raíz (/) y la videollamada en la ruta /webrtc.

Además, se ha añadido un botón "Videollamada WebRTC" en el reproductor que conecta directamente con la nueva funcionalidad, haciendo que las dos prácticas formen una sola aplicación coherente.

3. Arquitectura del sistema

La arquitectura del sistema sigue el patrón cliente-servidor combinado con comunicación peer-to-peer:



El servidor actúa únicamente como intermediario durante la fase de señalización. Una vez establecida la conexión WebRTC, el flujo multimedia viaja directamente entre los navegadores sin pasar por el servidor. El sistema se compone de los siguientes elementos:

- Servidor Node.js con Express: sirve los archivos estáticos y gestiona la señalización.
- Socket.IO: proporciona el canal WebSocket para la señalización.
- RTCPeerConnection: gestiona la conexión WebRTC entre los dos navegadores.
- RTCDataChannel: canal de datos peer-to-peer para el chat de mensajes.
- Apache: servidor web que escucha en el puerto 80 y hace de proxy hacia Node.js.
- pm2: gestor de procesos que mantiene Node.js activo en segundo plano.
- Groq (Whisper + LLaMA): servicio de IA para transcripción de audio y resumen de sesión.

4. Servidor de señalización

El servidor de señalización está implementado en Node.js utilizando Express y Socket.IO. Su función es coordinar la conexión inicial entre los dos navegadores.

El servidor gestiona los siguientes elementos:

- Usuarios conectados: cada vez que un socket se conecta, el servidor lo registra y le asigna un identificador único.
- Salas de comunicación: los usuarios se unen a salas identificadas por nombre. Cada sala admite un máximo de dos usuarios.
- Entrada y salida de usuarios: cuando un usuario entra a una sala llena, el servidor le notifica. Cuando un usuario se desconecta, el servidor avisa al otro participante.
- Reenvío de mensajes de señalización: el servidor recibe y reenvía los mensajes offer, answer e ice-candidate entre los dos peers.

El servidor no procesa ni almacena contenido multimedia. Su papel se limita a la coordinación inicial. El código está en el archivo server.js e incluye también dos endpoints REST para la IA:

- POST /api/transcribe: recibe un archivo de audio WebM, lo envía a Whisper de Groq y devuelve la transcripción.
- POST /api/ai-summary: recibe el log de la sesión y genera un resumen usando LLaMA 3.3 de Groq.

5. Uso de WebSockets

WebSocket es un protocolo de comunicación bidireccional cliente-servidor que mantiene una conexión persistente. En esta práctica se utiliza la librería Socket.IO, que proporciona una abstracción sobre WebSocket con soporte para eventos y salas.

Los mensajes que viajan por WebSocket son exclusivamente los de señalización:

UNIVERSITAT DE LES ILLES BALEARS
Laboratori de Tecnologies de la Informació i Multimèdia (LTIM)

Es importante destacar que por WebSocket no viaja ningún contenido multimedia. El audio, el vídeo y los mensajes de chat se transmiten directamente entre navegadores mediante WebRTC.

6. Uso de WebRTC

WebRTC (Web Real-Time Communication) es una tecnología que permite la comunicación multimedia directa entre navegadores sin necesidad de un servidor intermediario para el flujo de datos. En esta práctica se utilizan los siguientes elementos:

- RTCPeerConnection: gestiona la conexión peer-to-peer. Se configura con servidores STUN y TURN para permitir la conexión entre dispositivos en distintas redes.
- getUserMedia: captura el audio y el vídeo del dispositivo del usuario antes de iniciar la llamada.
- Tracks de audio y vídeo: se añaden a la RTCPeerConnection con addTrack() y se reciben en el otro peer mediante el evento ontrack.
- Servidores STUN de Google (stun:stun.l.google.com:19302) para descubrir la IP pública.
- Servidores TURN de Metered (turn:global.relay.metered.ca:443) para retransmitir el tráfico cuando los peers no pueden conectarse directamente.

7. Flujo join → offer → answer → ICE

El proceso de establecimiento de la conexión WebRTC sigue estos pasos:

1. Join

El primer usuario entra a la sala. El servidor lo registra y espera. El segundo usuario entra a la misma sala. El servidor detecta que hay dos participantes y emite el evento ready al segundo usuario.

2. Offer

El segundo usuario crea una `RTCPeerConnection`, añade sus tracks de audio y vídeo, crea el `RTCDataChannel` y genera una oferta SDP mediante `createOffer()`. Esta oferta describe sus capacidades multimedia. La establece como descripción local con `setLocalDescription()` y la envía al servidor, que la reenvía al primer usuario.

3. Answer

El primer usuario recibe la oferta, crea su propia `RTCPeerConnection`, añade sus tracks, establece la oferta recibida como descripción remota con `setRemoteDescription()`, genera una respuesta SDP con `createAnswer()`, la establece como descripción local y la envía al servidor, que la reenvía al segundo usuario.

4. ICE

Mientras se produce el intercambio de oferta y respuesta, ambos peers generan candidatos ICE. Cada candidato representa una posible ruta de red para conectarse. Los candidatos se envían al servidor mediante el evento `ice-candidate` y el servidor los reenvía al otro peer, que los añade con `addIceCandidate()`. Cuando se encuentra una ruta compatible, la conexión WebRTC queda establecida y el flujo multimedia viaja directamente entre los navegadores.

8. Diferencia entre WebSocket y WebRTC

La diferencia fundamental entre las dos tecnologías es el modelo de comunicación:

- WebSocket es una comunicación cliente-servidor. Todos los mensajes pasan obligatoriamente por el servidor. Es adecuado para mensajes de coordinación, pero no para flujos multimedia intensivos.
- WebRTC es una comunicación peer-to-peer. Una vez establecida la conexión, los datos van directamente de un navegador a otro sin tocar el servidor. Esto permite transmitir audio y vídeo en tiempo real con baja latencia.

En esta aplicación los dos coexisten con roles diferentes: WebSocket gestiona la señalización inicial (los primeros segundos de la llamada) y WebRTC gestiona todo el flujo multimedia una vez que la conexión está establecida.

9. RTCDataChannel

El `RTCDataChannel` es un canal de datos peer-to-peer incluido en la especificación WebRTC. Permite enviar texto o datos binarios directamente entre los dos navegadores sin pasar por el servidor.

En esta práctica se ha implementado un chat de mensajes usando RTCDataChannel. El canal se crea antes de generar la oferta SDP. El peer que recibe la oferta obtiene el canal mediante el evento ondatachannel. Los mensajes se envían con dataChannel.send(msg) y se reciben mediante channel.onmessage.

El uso de RTCDataChannel para el chat demuestra que los datos también pueden viajar por WebRTC directamente entre peers, no solo el audio y el vídeo.

10. Uso e implementación de IA

Transcripción de audio con Whisper

Durante la videollamada, el navegador graba el audio del micrófono local en segundo plano utilizando la API MediaRecorder. Al pulsar el botón "Transcribir y resumir sesión", la grabación se detiene y el audio se envía al servidor en formato WebM. El servidor lo reenvía al modelo whisper-large-v3 de Groq, que devuelve la transcripción en texto.

Resumen de sesión con LLaMA

Una vez obtenida la transcripción, el servidor construye un log completo de la sesión que incluye los estados de conexión, los mensajes del chat y la transcripción de audio. Este log se envía al modelo llama-3.3-70b-versatile de Groq con un prompt que le pide generar un resumen estructurado en español.

El resultado se muestra en el panel de IA de la interfaz. Si la API de Groq no está disponible, el sistema genera automáticamente un resumen local con la información disponible. La IA no sustituye a WebRTC ni a Socket.IO, actúa como capa de análisis posterior a la sesión.

11. Problemas encontrados y soluciones

Socket.IO no accesible desde el exterior

El servidor Node.js corre en el puerto 3000, pero Apache escucha en el puerto 80. Al principio Socket.IO no era accesible desde otros dispositivos.

Solución: se configuró Apache como proxy inverso añadiendo directivas ProxyPass y reglas de reescritura para WebSocket en el archivo de configuración de Apache.

Conexión WebRTC fallaba entre dispositivos en distintas redes

Con solo servidores STUN, la conexión fallaba cuando los dispositivos estaban en redes distintas con NAT restrictivo.

Solución: se añadieron servidores TURN de Metered al ICE_CONFIG. Los servidores TURN retransmiten el tráfico WebRTC cuando los peers no pueden conectarse directamente.

La red de la UIB bloquea el tráfico hacia servidores TURN externos

Al hacer pruebas desde el WiFi de la universidad, el ordenador no generaba candidatos ICE de tipo relay, lo que impedía la conexión con dispositivos en otras redes.

Solución: la aplicación funciona correctamente cuando ninguno de los dos dispositivos está en la red de la UIB. Es una limitación de infraestructura de red, no del código.

Transcripción de audio incorrecta

Whisper devolvía transcripciones incorrectas cuando el audio era muy corto o había mucho ruido por el procesamiento de WebRTC.

Solución: se ajustó el intervalo de grabación de MediaRecorder y se añadió un reinicio de la grabación después de cada transcripción.

Archivos temporales acumulándose en el servidor

Los archivos WebM que se generaban durante la transcripción no se eliminaban correctamente.

Solución: se restauró la llamada a `fs.unlinkSync()` en el endpoint de transcripción para limpiar los archivos temporales después de cada uso.

12. Extensiones implementadas

RTCDataChannel — chat de mensajes peer-to-peer

Se ha implementado un canal de datos usando `RTCDataChannel` que permite el intercambio de mensajes de texto directamente entre los dos navegadores sin pasar por el servidor. Los mensajes del chat forman parte del log de sesión que posteriormente analiza la IA.

IA online — transcripción y resumen con Groq

Se han integrado dos modelos de IA online a través de la API de Groq:

- Whisper large v3: transcripción de voz a texto del audio local grabado durante la llamada.
- LLaMA 3.3 70B: generación de un resumen inteligente en lenguaje natural de la sesión completa.

Esta extensión demuestra cómo la IA puede actuar como capa de análisis sobre una sesión WebRTC, procesando tanto los mensajes de texto del chat como el audio de la conversación.

Indicador visual del micrófono

Se ha añadido un visualizador de nivel de audio en tiempo real usando la API `AudioContext` y `AnalyserNode` del navegador. El indicador muestra barras animadas que reflejan el volumen del micrófono durante la llamada, facilitando la detección de problemas de audio.